

HomeBench: 在智能家居中通过单设备与多设备场景下的有效指令和无 效指令评估大语言模型

Silin Li¹, Yuhang Guo¹, Jiashu Yao¹, Zeming Liu^{2*}, Haifeng Wang³ 北京
理工大学计算机科学与技术学院²北京航空航天大学计算机科学与工程学院³百度
公司 {lisilin, guoyuhang, yaojiashu}@bit.edu.cn zmliu@buaa.edu.cn
wanghaifeng@baidu.com

摘要

大语言模型 (LLMs) 有潜力通过增强其准确理解用户需求并做出适当响应的能力, 来革新智能家居助手, 这对于构建更智能的家庭环境极为有益。尽管最近的研究探索了将大语言模型集成到智能家居系统中, 但它们主要侧重于处理简单、有效的单设备操作指令。然而, 现实场景要复杂得多, 通常涉及用户发出无效指令或同时控制多个设备。这带来了两大挑战: 大语言模型必须准确识别并纠正用户指令中的错误, 并完美执行多个用户指令。为了应对这些挑战并推动基于大语言模型的智能家居助手的发展, 我们在本文中引入了 HomeBench, 这是首个包含单设备和多设备场景下有效及无效指令的智能家居数据集。我们在13个不同的大语言模型上进行了实验; 例如, GPT-4o在无效多设备指令场景下的成功率仅为0.0%, 这表明即使借助上下文学习、检索增强生成和微调, 现有的最先进大语言模型在此类情况下仍表现不佳。我们的代码和数据集已在链接¹公开提供。

1 引言

将大语言模型 (LLMs) 集成到智能家居助手中, 不仅能帮助用户以更简单、更少的指令控制设备, 还能基于用户的习惯和历史行为实现自动设备控制, 而无需明确的指令 (King等人, 2024年; Rivkin等人, 2024年; Yin等人, 2024年)。例如, King等人 (2024年); Rivkin等人 (2024年) 引入了



图1: 有效单设备指令、无效单设备指令、有效的多设备指令、无效多设备指令和混合多设备指令的示例。橙色表示待操作的设备, 蓝色表示待操作设备的方法, 红色表示待操作的错误设备。这些错误输出是由于操作了不存在的设备或不存在的设备功能, 导致模型产生幻觉并操作了错误的设备。

Sasha和Sega系统, 初步验证了将大语言模型集成到智能家居助手中的可行性。Yin等人 (2024年) 进一步探索了大语言模型在处理智能家居控制中模糊和无指令场景的能力, 为大语言模型在该领域的应用前景提供了更多证据。这些研究主要关注有效单设备指令的操作 (King等人, 2024年; Rivkin等人, 2024年; Yin等人, 2024年), 并且在这些研究中, 大语言模型能够控制的设备范围和数量都相当有限。

然而, 在现实场景中, 用户可能因各种原因输入无效指令, 例如醉酒或口误。如果模型

*通讯作者: Zeming Liu¹

<https://github.com/BITHLP/HomeBench>

忠实地执行这些无效指令，可能会对用户体验和安全性产生负面影响。如图1所示，错误地激活厨房电器或解锁门锁可能带来重大的安全风险。此外，在涉及多设备的现实场景中，用户通常需要同时控制多个设备。因此，开发能够全面管理多设备、理解复杂环境中的动态交互，同时优化对潜在错误的警告与纠正机制的大语言模型，成为推动智能家居系统发展的关键因素。

为了弥合这些差距，我们引入了一个新的综合性评估基准：HomeBench，这是首个涵盖单设备和多设备有效指令与无效指令的智能家居数据集，其设备种类多样且数量庞大。此外，为了便于灵活的试验和场景扩展，我们开发了一个可定制的虚拟家居环境，允许灵活设置各种设备和操作场景，为模型测试与优化提供了一个多功能的实验基础。具体而言，我们构建了100个智能家居场景，覆盖超过17万条用户操作指令。每个场景至少包含47个智能设备，涵盖至少10种不同类型。针对多设备操作场景，我们模拟了涉及1到10个设备的复杂交互，以评估模型在高度复杂环境中的性能。通过引入HomeBench，我们旨在为下一代智能家居助手铺平道路，使其能够在现代物联网生态系统中无缝管理和协调众多智能设备。

我们的主要贡献可总结为以下三点：

- 据我们所知，我们是首个研究模型在智能家居中处理无效指令及多设备操作指令能力的工作。
- 我们提出了HomeBench，它包含针对单设备和多设备的有效指令与无效指令，覆盖复杂的操作场景，并开发了一个可定制的虚拟环境以支持数据集的生成与扩展。
- 我们对13种不同大语言模型的实验结果表明，几乎所有模型在此基准测试中表现

都不佳，尤其是在处理无效指令和多设备指令方面。进一步分析表明，简单的上下文学习和微调能显著提升性能，但它们与实际应用场景之间仍存在差距。

2 相关工作

2.1 大语言模型之前的家庭助理

智能家居系统由相互连接的物联网智能设备构建而成，用于监测、感知和控制家庭环境。自动化这些设备不仅能提升生活质量和舒适度，还能优化资源效率（Ki等人，2020年）。近期的研究侧重于通过机器学习技术增强智能家居系统的功能。例如，Jaihar等人（2020年）开发了一种深度学习算法，利用加速度计数据识别用户行为以实现家庭自动化。Rani等人（2017）专注于基于语音的家居助手，其设计旨在理解用户的语音指令。借助先进的自然语言处理技术，诸如Bixby、Google Assistant和Alexa等商业解决方案提供了用户友好的界面，能够处理从购物、设置提醒到设备控制 and 家庭自动化等多种指令和查询。然而，这些现代家庭助理在处理模糊或复杂指令方面仍面临挑战（Luger和Sellen，2016）。

2.2 大语言模型上的家庭助理

为了克服其中一些挑战，近期研究提出利用大语言模型的推理能力来更好地理解和执行用户指令。Shi等人(2024a) 提出了AwareAuto系统，该系统标准化了用户表达，并采用由大语言模型驱动的两步推理方法来生成自动化解决方案，在符合用户意图方面达到了91.7%的准确率。Civitarese等人(2024年) 开发了ADL-LLM，它将原始传感器数据转化为文本描述，以实现日常生活活动的零样本识别。GestureGPT (Zeng等人,2024年) 将大语言模型的推理能力集成到一个三智能体框架中，实现了手势分析和智能家居任务的执行。Sasha (King等人,2024年) 展示了大语言模型在智能家居环境中的应用，证明了其有效响应复杂或模糊

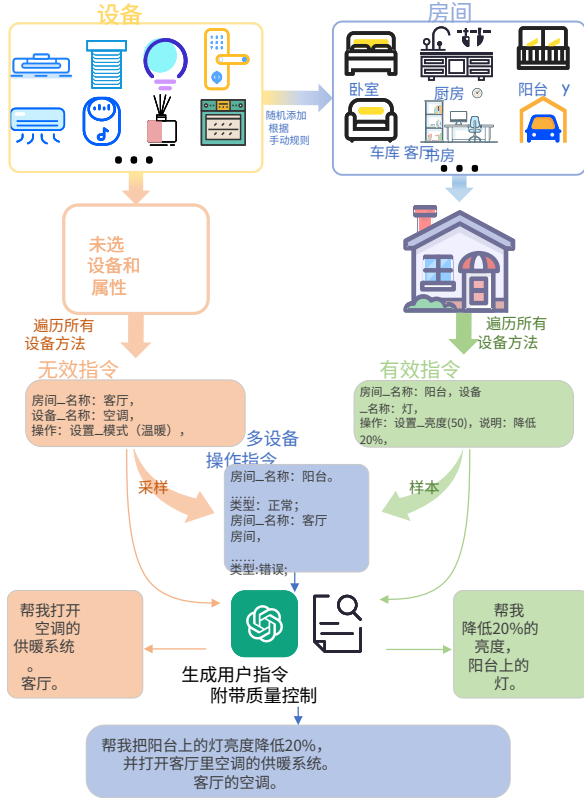


图2：数据集HomeBench的完整收集流程，涵盖设备构建、房间设置、指令生成、用户指令合成及质量控制。

指令的能力。Sasha实现了一个决策过程，其中每个步骤（例如，设备选择或例行程序验证）都由大语言模型支持。SAGE系统通过动态构建提示词树并采用大语言模型控制的离散操作序列，解决了大语言模型在缺乏关于用户和家庭的具体知识方面的局限性。这显著增强了其在信息检索、用户交互和设备操作方面的能力。然而，这些研究忽略了用户无效指令和多个操作指令的真实场景。

3 数据集收集

HomeBench 旨在构建高质量的智能家居指令调用数据，以提升在复杂家庭环境中针对单设备和多设备的调用能力，重点关注大语言模型在面对用户无效指令时的准确响应能力。在本节中，我们将描述数据集的任务格式，然后解释如何高效且有效地构建数据集。图2 展示了收集数据集的完整流程。

HomeBench。

3.1 任务定义

给定一条用户指令 u 和一个包含房间信息的虚拟家庭环境 $H = \{r_1, r_2, r_3, \dots\}$ ，每个房间 r_i 包含当前房间内的一组可操作设备 $D = \{d_1, d_2, \dots\}$ 。每个设备 d_j 包含其可操作属性的状态 s 以及一组可调用方法 $M = \{m_1, m_2, m_3, \dots\}$ 和相应的参数。大语言模型需要根据用户指令 u ，决定调用哪个房间中哪个设备的哪个方法，以改变目标设备的属性，即 $h, u \rightarrow [r_i.d_j.m_k]$ 。如果用户输入的是无效指令，则输出为错误_输入。

3.2 数据分类

数据集	VS IS	VM IM	MM SIZE
IFTTT (Yu等人, 2021)	✗	✗	✗
SEGA (Rivkin等人, 2024年)	✓	✗	✗
家庭助理 ²	✓	✗	✗
Ours	✓	✓	✓

表1：与现有数据集在指令类别上的比较，以确保公平对比。"✗"代表用于智能家居操作的自动化指令生成，这与我们的任务设定有显著不同。

根据指令类型和涉及的设备数量，我们可以将数据分为五种不同的类型。每种类型都对应现实场景中的典型用例，为评估大语言模型调用智能家居设备的能力提供了一个全面的基准：

- 有效单设备指令 (VS)：用户发出的指令涉及单一设备的单一方法。该指令正确且可执行。
- 无效单设备指令 (IS)：用户发出的指令涉及单一设备的单一方法。然而，该指令不正确且无法执行。
- 有效的多设备指令 (VM)：用户发出涉及多个设备上多种方法的指令。所有指令均正确且可执行。

²
<https://huggingface.co/datasets/acon96/Home-Assistant-Requests>

统计信息		VS	IS	VM	IM	MM	ALL
训练集	平均值指令长度	11.13	11.27	32.48	27.82	69.37	25.59
	平均操作设备数量	1.00	1.00	2.91	2.51	6.26	2.30
	指令数量	49,048	53,771	1,829	826	33,443	138,917
有效	平均指令长度	11.10	11.29	34.37	26.88	69.30	25.45
	平均操作设备数量	1.00	1.00	3.12	2.40	6.26	2.26
	指令数量	6,145	6,731	232	124	4,132	17,364
Test	平均指令长度	11.19	11.28	33.49	25.88	68.81	25.13
	平均操作设备数量	1.00	1.00	2.98	2.30	6.21	2.29
	指令数量	6,187	6,765	245	97	4,072	17,366

表2: 我们提出的HomeBench的数据统计。

- 无效多设备指令 (IM)：用户发出涉及多个设备上多种方法的指令。然而，所有指令均不正确且无法执行。

- 混合多设备指令 (MM)：用户发出涉及多个设备上多种方法的指令。其中一些指令正确且可执行，而另一些指令错误且无法执行。

表1展示了数据集HomeBench 与其他流行数据集的详细对比。目前，大多数基准测试侧重于处理单个设备的有效指令，而现实世界中常见的多设备操作指令和无效指令往往被忽视。本研究在这两个方面做出了重要贡献。

3.3 数据收集

考虑到市场上访问智能家居开发环境的挑战，我们采用了虚拟智能家居设置，以降低我们研究的入门门槛。数据集构建分为两个主要阶段：构建虚拟设备和环境，以及生成指令。

对于虚拟设备构建，我们收集了15种常用的智能家居设备，并为每种设备识别了可通过智能家居平台调用的API方法。例如，扫地机器人等设备。我们使用Python类实现了这些虚拟设备，允许通过API方法调用来修改属性。为了考虑不同品牌和型号之间的性能差异，我们简化了部分设备的方法，仅保留核心的智能家居功能。对于其他

设备，我们随机添加了额外的方法，以创建一个多样化的设备池。为了增加任务的复杂性，我们设计了十二个房间，每个房间都有独特的功能。设备从池中随机选择，并根据手动定义的规则分配到各个房间。

为了确保数据质量的同时最小化人工工作量，我们使用GPT-3.5来生成用户指令（Wang等人, 2024d）。对于有效指令，我们首先在虚拟环境中创建了所有可能的设备操作命令，指定了房间、设备名称、调用的方法以及方法的输入参数。借助提示规则和少样本示例，我们将这些信息提供给ChatGPT以合成用户指令。对于无效指令，我们应用逆向规则来识别虚拟房间中不存在的设备或缺失的方法，并生成相应的错误操作命令。这些指令也通过相同的方法由ChatGPT处理。对于涉及多条指令的场景，我们从有效操作池和无效指令池中随机选择命令，然后使用相同的合成方法来生成用户指令。

为确保数据质量，我们从每个智能家居场景中生成的用户指令中抽样10%，并从三个维度进行了严格评估：流畅性检查（Wang et al., 2024b; Liu et al., 2020年）（语言清晰度）、语义对齐（Iskander et al., 2024年）（指令与设备/房间能力对比）以及动作一致性（Iskander et al., 2024年）（合成指令与可执行机器指令对比）。任何不匹配都会导致立即拒绝（得分=0）。如果一条指令得分超过4分，则该场景的数据被视为有效。此

过程重复进行，直至收集到100个高质量智能家居场景数据。

3.4 数据统计与分析

表2 展示了数据集的统计数据。遵循先前的工作 (Zhang et al., 2023年; Kweon et al., 2023年; Chen et al., 2022年)，我们按照常规的8:1:1比率划分数据集，并提供了关于平均指令长度、指令操作设备数量以及根据VS、IS、VM、IM和MM数据类型的指令数量的详细信息。我们的数据分布揭示了两个值得解释的显著模式：（1）单设备指令比多设备指令更为普遍，（2）IM指令的数量相对较少。下面我们为这些观察提供详细解释：

- **单设备指令数量 > 多设备指令数量：**从负载均衡理论的角度来看，人类倾向于多次执行单一类型的任务，而不是同时执行多个不同的任务 (Sweller, 2011年)。因此，单设备操作命令的出现频率显著高于多设备操作命令。
- **即时消息指令数量较少：**从概率角度来看，在多个设备上执行完全有效或完全无效操作的可能性远低于混合有效和无效操作的可能性。

为了展示我们任务的复杂性，我们统计了100个家庭中设备的类型和数量。在12个不同功能的房间中，至少有47个设备可被操作，且至少有10种类型的设备可被操作。更多分析请参见附录A.1。

4 实验

4.1 设置

模型。受Fei等人; Huang等人的启发，我们从开源和闭源模型中选择了若干大语言模型，旨在提供一个全面的评估。具体而言，我们选择了Mistral-7B (Mistral-7B-v0.3) (Jiang等人, 2023年)、LLaMa3-8b (llama3-8b-Instruct) (AI@Meta, 2024年)、Qwen系列 (Qwen2.5-7B/32B/72B-Instruct) (Yang等人, 2024年)、Gemma系列 (gemma-2-9b/27b-it) (Team等人, 2024年) 作为开源大语言模型。

除此之外，我们还从闭源大语言模型中选择了GPT4 (GPT-4o) 和deepseek v3 (DeepSeek-AI等人, 2024年)。此外，我们还测试了面向推理的模型，如O1、O3 (o1-mini, o3-mini) (OpenAI等人, 2025)、DeepSeek-R1 (DeepSeek-AI等人, 2025) 以及QWQ-32B (Team, 2025)。我们构建了两类基线模型用于实验比较：一种基于提示词，另一种基于少样本上下文学习 (Brown等人, 2020年; Zhou等人, 2024年)。如第5.1节所示，模型在3样本后的性能提升有限，因此我们在实验中选择了4样本。在这两种类型的基线模型中，我们将虚拟家庭中每个设备的状态信息以及设备可以调用的方法嵌入到提示词中，为模型提供上下文信息。此设置旨在确保两种方法在相同的信息条件下进行评估，以全面比较其性能。具体细节请参阅附录A.2。

4.2 评估指标

在本研究的实验设计中，我们遵循具身智能领域的标准化测试范式，采用虚拟环境作为测试平台 (Zhu等人, 2024年; Zhang等人, 2024b)。基于功能等价性原则，虚拟设备与物理设备在API接口层面保持完全一致的测试标准：当模型正确生成符合预定义规范的API调用序列时，即判定测试通过。值得注意的是，虚拟测试环境具有独特的实验优势，它消除了物理设备固有的机械延迟 (无需等待现实世界的反馈周期)，从而显著提升了测试效率。

为评估大语言模型在执行用户指令方面的有效性，我们选取了两个关键指标：指令执行成功率 (Shi等人, 2024a; Rivkin等人, 2024年) 和F1分数。

成功率 (Succ)：我们遵循Shi等人; Rivkin等人的方法，该指标用于衡量用户指令中的所有设备操作是否均成功执行。

F1分数 (Li等人, 2024年; Cai等人, 2024年; Wang等人, 2024c)：我们首先计算精确率P，其定义为模型正确预测的操作数量除以模型预测的

模型	VS		IS		VM		IM		MM		ALL	
	SUCC	F1	SUCC	F1	SUCC	F1	SUCC	F1	SUCC	F1	SUCC	F1
Mistral-7B	2.49	4.68	0.01	0.03	0.82	6.23	0.00	0.00	0.00	2.26	0.90	2.33
LLaMA3-8B	47.11	47.36	0.61	0.61	3.27	31.83	0.00	0.52	0.00	15.20	17.07	18.21
Qwen2.5-7B	30.40	31.92	0.00	0.00	13.69	33.81	0.00	0.00	0.00	16.13	11.02	16.24
Qwen2.5-32B	41.28	41.34	14.04	16.34	14.29	26.33	0.00	9.55	0.27	19.45	20.44	23.06
Qwen2.5-72B	42.90	42.91	11.47	12.57	19.59	38.06	1.03	9.07	0.15	24.79	20.07	25.92
Gemma2-9b	12.32	12.58	0.00	0.02	0.41	11.44	0.00	0.98	0.00	6.40	4.39	6.42
Gemma2-27B	12.32	13.08	0.93	1.03	0.82	11.18	0.00	2.33	0.02	6.93	4.77	7.34
Deepseek-v3	71.08	71.08	39.05	38.40	45.31	69.14	0.00	11.11	0.50	28.15	41.19	38.81
GPT-4o	46.74	46.76	79.07	78.90	13.88	26.74	0.00	55.19	2.09	28.43	48.14	42.70
QWQ-32B	63.39	63.60	7.35	11.05	6.95	15.88	12.35	30.00	3.54	32.20	26.53	35.30
DeepSeek-R1	65.67	65.76	83.74	83.76	0.00	1.95	0.00	23.22	1.41	23.22	56.39	40.96
o1-mini	38.65	39.46	20.34	30.34	1.19	2.09	0.00	27.75	0.73	15.13	21.85	23.20
o3-mini	27.57	28.16	45.43	57.51	4.56	13.64	0.00	33.08	0.21	16.30	27.66	27.32
Mistral-7B-ICL	47.10	53.63	0.15	0.17	17.96	50.07	0.00	0.00	0.02	22.74	17.10	24.15
LLaMA3-8B-ICL	68.08	68.17	2.38	2.37	30.61	60.63	0.00	2.71	0.17	28.69	25.65	30.78
Qwen2.5-7B-ICL	66.96	66.89	14.07	14.08	37.55	69.18	0.00	5.82	0.49	33.23	29.98	35.72
Qwen2.5-32B-ICL	77.24	77.48	69.33	69.27	45.31	73.46	3.09	44.44	5.38	52.71	56.44	59.84
Qwen2.5-72B-上下文学习	82.27	82.07	35.77	35.78	51.02	76.05	6.19	27.89	2.95	48.42	44.60	52.01
Gemma2-9B-上下文学习	59.72	59.65	21.67	21.74	33.47	62.54	3.09	20.62	1.55	42.17	30.57	41.67
Gemma2-27B-上下文学习	79.44	79.50	11.06	11.07	48.16	74.71	1.03	11.21	0.81	41.97	33.48	43.00
Deepseek-V3-上下文学习	80.00	80.06	58.58	58.48	54.69	77.29	11.34	32.22	5.35	52.74	53.41	58.74
GPT-4o-上下文学习	74.25	74.30	87.10	87.09	45.71	71.48	61.86	81.18	23.35	78.98	66.85	79.58
QWQ-32B-上下文学习	67.71	64.50	79.32	79.30	35.51	61.97	65.98	79.39	18.71	70.02	60.28	70.50
Deepseek-R1-ICL	76.48	77.46	84.18	84.77	46.19	70.75	66.27	83.03	28.28	77.73	67.62	78.82
o1-mini-ICL	75.42	75.94	86.80	86.90	53.39	75.52	79.17	89.09	32.25	81.37	69.47	81.42
o3-mini-ICL	83.77	84.25	88.36	88.38	57.51	80.49	64.04	85.19	38.49	85.57	74.44	85.75

表3：不同大语言模型在HomeBench上的主要结果。**粗体**表示最佳性能

操作总数。

$$Precision = \frac{\text{operation_correct_num}}{\text{operation_pred_num}} \quad (1)$$

我们计算召回率R，即模型正确预测的操作数量除以用户指令中实际所需操作的总数。

$$Recall = \frac{\text{operation_correct_num}}{\text{operation_gold_num}} \quad (2)$$

F1分数按常规公式计算，为 $2*PR / (P+R)$ 。

4.3 结果

表 3 分别展示了不同大语言模型在 HomeBench 上针对不同类型用户指令的结果。从结果中可以得出若干结论。

总体而言，**o3-mini** 取得了最佳的整体性能，而 **o1-mini** 在某些情况下表现优于 **o3-mini**，尤其是在仅处理无效指令时。一般来说，除了 Deepseek-V3、GPT-4o 和推理模型外，其他模型在处理各类指令方面均显著落后于 o3-mini。这一现象可归因于我们任务的核心难点

在于其涉及长上下文注意力。模型需要全面解读一整套设备状态和方法，以识别可操作设备和那些不存在的设备。在评估的模型中，o3-mini 展现出明显更优的长距离上下文管理能力。尽管大语言模型已取得显著进展，但现有模型在处理复杂规划任务时仍然存在困难，尤其是当这些任务涉及需要多设备操作的混合指令时。事实上，即使在多模态任务中加入了上下文学习，o3-mini 的成功率也仅为 38.49%。

随着模型大小的增加，我们观察到在不同类型的指令上性能有所提升。这种提升在 Qwen2.5和Gemma2系列模型中尤为明显。通常，更大的模型往往能提供更优的性能。然而，在存在输入无效指令或多设备指令的情况下，大多数模型的性能显著下降，部分模型在特定任务（如指令修改任务）上的SUCC甚至为0%，这表明在完全识别错误方面仍存在挑战。即使是像Deepseek-v3这样的先进模型，也只能部分检测到指令中的错误。此外，

即使将上下文学习纳入任务设置，这些问题依然存在，大多数模型难以克服指令修改任务带来的挑战。

随着上下文学习的引入，模型在处理各种指令任务方面展现出显著的性能提升。这一现象在所有测试模型中均得到验证，部分模型甚至取得了突破性的性能增益。例如，Mistral模型在指令选择任务中实现了超过10倍的改进，而Gemma2-9b模型的指令选择任务成功率从0%跃升至21.67%。即使在指令修改和多模态任务等更复杂的任务中，性能提升也相当可观。值得注意的是，Gemma2-9b模型在这些任务中取得了20个百分点的增长。然而，尽管有这些显著增强，模型性能与实际应用需求之间仍存在相当大的差距，凸显了进一步优化和完善的必要性。

模型的性能深受指令复杂性的影响。分析不同场景下的性能表现揭示了以下趋势：视觉选择任务>虚拟机任务>指令选择任务>多模态任务>指令修改任务。这一趋势在大多数大语言模型中保持一致，包括Qwen2.5、Gemma2、LLaMA3-8B、Deepseek-v3和Mistral-7B。这与我们的预期相符，因为指令修改任务最为复杂，其次是多模态任务和指令选择任务，而虚拟机任务相对简单，视觉选择任务则最容易。然而，这一趋势并不适用于Deepseek-V3、GPT-4o和推理模型。关键原因在于指令选择任务和指令修改任务的性质，它们需要强大的长上下文注意力。在这些任务中，模型必须跟踪所有设备的状态和方法，以确定哪些操作是有效的，哪些设备不可用。Deepseek-V3、GPT-4o和推理模型在上下文处理能力方面优于其他开源模型（Grattafiori等人，2024年；Wang等人，2024a；Zhang等人，2024c）。

5 分析

在本节中，我们进行全面的分析，旨在回答三个研究问题。RQ1：少样本如何影响模型性能？

（见第5.1节）RQ2：是否有必要先识别房间和设备以减小上下文窗口？（见第5.2节）以及RQ3：当前大语言模型在我们任务中的主要瓶颈是什么（见第5.3节），微调能否缓解它？（见第5.4节）。

5.1 不同上下文学习样本数量的影响

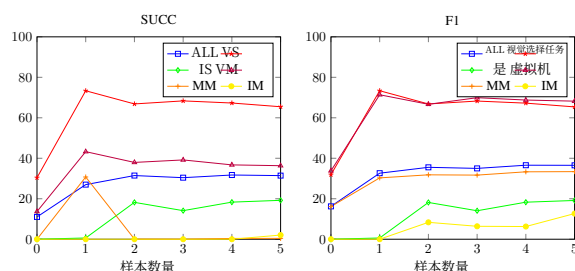


图3：Qwen2.57B-Instruct 模型在添加不同类型数据样本时的性能（添加顺序为：视觉选择任务、是、虚拟机、多模态任务、即时消息）。

图3展示了Qwen2.5-7B-Instruct在使用不同演示样本（shots）时的性能表现，其他模型的性能见附录A.2。由于我们的数据类型相对丰富，每次添加的样本类型也不同。具体的添加顺序为：VS、IS、VM、MM、IM。这里的样本是通过在训练集中随机抽样获得的。详细的抽样方法和信息，请参阅附录A.2中的描述。实验结果表明，当我们添加不同类型的样本时，模型的整体效果在一定程度上得到提升，相应的数据类型也表现出不同程度的改进。例如，当添加VS（1-shot）时，模型在VS上的成功率从30.40%提升至73.35%；而当添加IM（5-shot）时，模型在IM上的F1分数从6.27%提升至12.67%。这些结果表明，适当增加样本的数量和类型有助于提升模型在特定数据集上的性能。然而，我们也观察到，随着样本数量的增加，某些任务的性能出现下降，尤其是IS和VS。这可能是因为提供的样本主要关注用户指令和设备操作，而缺乏关于家庭设备当前状态的信息。因此，模型难以准确处理这些任务，导致性能下降。因此，虽然增加样本数量通常能提升整体性能，但确保样本内容的完整性和相关性至关重要。未能包含关键的上下文信息可能会引入信息偏差，对模型在特定任务中的有效性产生负面影响。

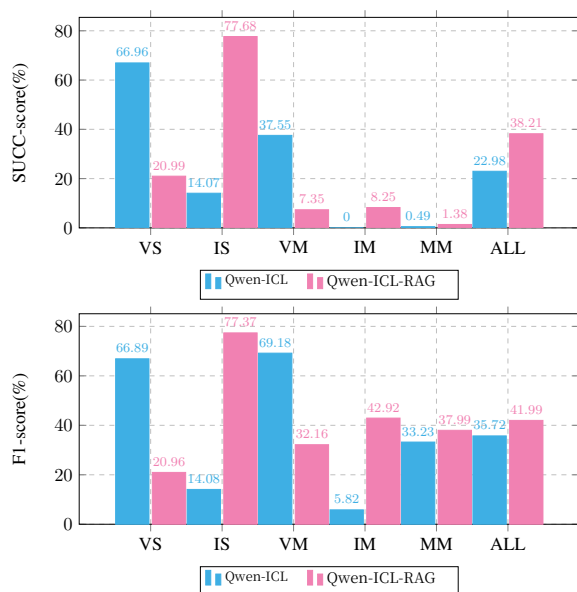


图4: Qwen-ICL与Qwen-ICL-RAG之间的性能差距。

5.2 检索增强生成的效果

在我们的任务设置中，需要在提示词中提供智能设备的状态信息和可调用方法，这导致了较高的令牌数，超过3k个令牌。然而，每次操作通常只涉及1到10个设备，使得大量信息变得冗余。为了减少冗余并提升整体性能，我们尝试使用检索增强生成（RAG）进行优化（Lewis等人, 2020年; Wang等人, 2024e; Shi等人, 2024b）。我们根据房间信息对上下文进行了分段，检索出与当前任务最相关的房间并提供给大语言模型。具体的设置细节可在附录A.2中找到。结果如图4所示。可以观察到，实验结果出现了异常：模型原本擅长的任务性能有所下降，而它不太擅长的任务则表现出显著提升。我们对结果进行了错误分析，发现检索到的房间包含大量错误。尽管上下文缩短了，但有用信息的数量也随之减少，导致涉及有效指令的任务性能下降，而涉及无效指令的任务性能则有所提升。我们在表4中添加了检索上下文错误的统计分析，并在附录A中提供了检索到错误上下文时的输出案例。

	单设备指令	多设备指令
无帮助	74.49%	46.47%
状态上下文		
无有用信息	75.97%	48.74%
方法上下文		
缺乏有用信息	-	46.65%
状态上下文		
缺乏有用	-	45.04%
方法上下文		

表4: RAG上下文检索中的错误统计。我们进行了统计分析，以确定使用嵌入向量相似度为Qwen模型检索的上下文是否与给定输入指令所需的上下文一致。

5.3 错误分析

错误类型	输入	生成内容和黄金标准	比率
不忠实性	... master_卧室: 灯: 状态: 关闭 ... <用户指令: > 请将 主卧室的灯调至50%。	错误_输入 主卧_室.灯。 设置_亮度(50)	27.93
上下文内 注意力错误	... 客厅_: 媒体_播放器: 状态: 开启 音量: 28 ... <用户指令: > 降低音量 媒体播放器的音量 阳台调高3%。	客厅_房间.媒体_播放器.设置_音量(20) 客厅_房间.媒体_播放器.设置_音量(25)	43.39
关键错误	... 商店_房间: (没有空气净化器) ... <用户指令: > 帮我把储藏室的空气 净化器调到强力模式。 房间切换至强力模式。	store_房间.airpurifiers.set_模式(high) error_输入	34.39
	... guest_房间: 供暖: 状态: 开启 模式: 供暖 ... <用户指令: > 将客房中的供暖 设置为仅风扇模式。	客房_卧室.供暖。 设置_模式("仅_风扇") 客_卧室.供暖 .设置_模式(仅_风扇)	

表5: GPT-4o测试结果中的三种主要错误类型分别是不忠实性、上下文注意力错误和关键错误。红色代表生成内容绿色代表黄金标准答案。由于多设备操作会生成多条指令，一条用户指令可能包含多种错误类型。我们还在附录A.2中提供了针对各种数据类型的不同错误类型的比例。

我们对实验结果进行了错误分析，以识别当前最佳模型GPT-4o的潜在瓶颈。如表5所示，错误可分为以下三种类型：

- 1) 不忠实性：模型未能准确执行用户指令，错误地将一个

正确的指令判定为不正确。例如，在表格所示的案例中，模型仅因为主卧灯已关闭，就错误地认为一盏灯无法操作，从而给出了错误的响应。

2) 上下文注意力错误：模型未能注意到上下文中包含的关键信息，导致输出无效指令。例如，表格中的用户指令是调整相对音量，但模型未能准确掌握当前设备的状态信息。另一个问题是，模型未能全面考虑所有设备的状态和方法，导致无法确认哪些设备是可操作的或不存在的。

3) 关键错误：模型的输出不遵循提示词指定的格式，导致无法对结果进行有效的统计分析。

5.4 微调的效果

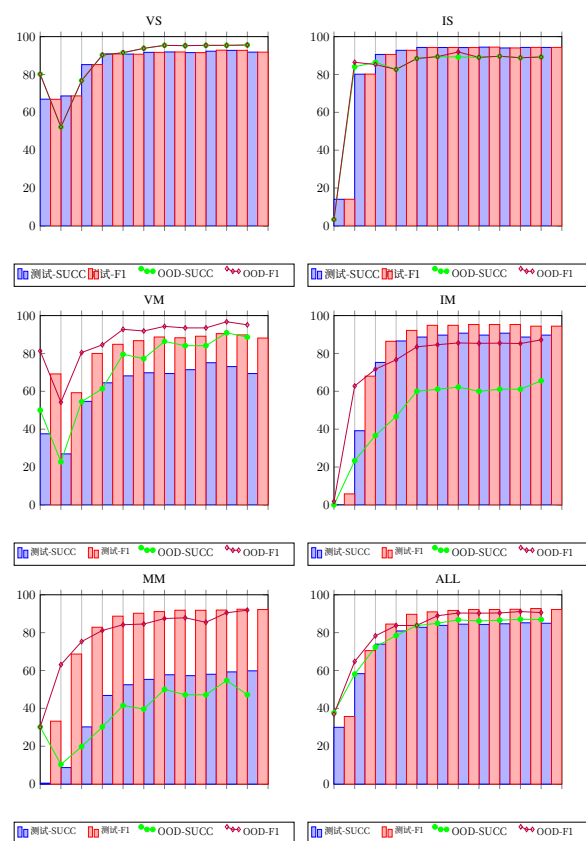


图5: Qwen2.5-7B-Instruct在测试数据集和OOD数据集中，不同训练步数下的性能表现。

我们使用LoRA在完整训练数据集上对Qwen2.5-7B-Instruct进行了2轮微调，每100个训练步数进行一次测试。测试结果如图5所示。随着训练步数的增加，各项

任务的性能均显著提升，但在训练1轮后，性能提升逐渐趋于平缓。最终，该模型在所有任务上的表现均超越了GPT-4o-ICL。对于除VM和MM之外的大多数任务，SUCC和F1分数均超过80%。然而，尽管经过训练，VM和MM的SUCC分数仍低于70%。为了进一步评估泛化能力，我们进行了分布外评估，引入了两个先前未见过的设备。结果显示，即使在这些新设备上，模型的性能也与其在测试集上的结果保持一致，这证明了我们数据集的鲁棒性和多样性。关于OOD数据的详细信息，请参阅附录A.2。然而，尽管微调带来了性能提升，但并未从根本上解决关键问题。更深入的分析表明，SUCC分数低的主要原因在于上下文注意力错误和不忠实性，这凸显了仍需进一步改进的领域。

6 结论

为了推动基于LLM的智能家居助手的发展，并应对现实场景中无效指令和多设备指令带来的挑战，我们在本文中提出了HomeBench，这是首个包含单设备和多设备场景下有效指令与无效指令的智能家居数据集。我们评估了13个大语言模型，并探索了包括检索增强生成、上下文学习和微调在内的多种方法来提升其性能。我们的工作为基于LLM的智能家居助手的优化与应用奠定了坚实的基础。

局限性

我们承认本工作的以下局限性。我们的数据集仅包含英语单语数据，而实际应用可能涉及多语言。在构建过程中，我们未考虑不同品牌设备之间的差异，尽管这种差异在实际应用中确实存在。我们相信，通常可以通过修改设备名称来区分这些应用。此外，品牌选择往往取决于个人偏好，这超出了本研究范围。

伦理声明

在研究过程中，我们已彻底审查并确保遵守伦理标准。我们构建的设备和房间均经过仔细筛选，并在数据合成过程中应用了严格的质量控制，以避免任何形式的冒犯性或偏置内容。因此，我们认为我们的研究不存在伦理问题。所用数据来源合乎伦理，分析公正，所有程序均遵循既定的伦理准则。

致谢

我们感谢所有匿名审稿人富有洞察力且宝贵的意见。本研究得到了国家自然科学基金（项目编号：U21B2009、62406015）和北京理工大学科技创新计划（项目编号：23CX13027）的支持。

参考文献

AI@Meta. 2024年. [Llama 3 模型卡](#).

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020年. Language models are few-shot learners. In第34届神经信息处理系统国际会议论文集, NIPS ' 20, Red Hook, NY, USA. Curran Associates Inc.

蔡宇翔、刘乔、甘阳磊、林润、李长林、刘学义、罗达、贾业阳。2024年. [DiFiNet: 用于嵌套命名实体识别的边界感知语义差分与过滤网络](#)。载于第62届计算语言学协会年会论文集（第一卷：长论文），第6455–6471页，泰国曼谷。计算语言学协会。

Liyang Cheng, Lidong Bing, Ruidan He, Qian Yu, Yan Zhang, and Luo Si. 2022. [IAM: 一个用于集成论证挖掘任务的大规模综合数据集](#)。载于《第60届计算语言学协会年会论文集（第一卷：长论文）》，第2277–2287页，爱尔兰都柏林。计算语言学协会。

Gabriele Civitarese, Michele Fiori, Priyanka Choudhary, and Claudio Bettini. 2024. [大语言模型是日常生活活动的零样本识别器](#)。预印本, arXiv:2407.01238.

DDeepSeek-AI、郭达雅、杨德建、张浩伟、宋俊骁、张若愚、徐润欣、朱启昊、马士荣、王培屹、毕啸、张晓康、于兴凯、吴宇、吴泽锋、勾志斌、邵志宏、李卓书、高子怡、刘爱新、薛冰、王炳轩、吴博超、冯蓓、陆成达、赵成刚、邓承启、张宸宇、阮冲、戴大麦、陈德礼、季东杰、李尔航、林方韵、戴福聪、Fuli Luo、郝广博、Guanting Chen、Guowei Li、H. Zhang、Han Bao、Hanwei Xu、Haocheng Wang、Honghui Ding、辛华健、高华祚、曲慧、李辉、郭建中、李佳实、王家伟、陈景昌、袁景阳、邱俊杰、李俊龙、蔡佳龙、倪佳琪、梁健、陈进、董凯、胡凯、高凯歌、管康、黄可欣、余快、王利安、张乐聪、赵亮、王立通、张立月、徐磊、夏乐怡、张明川、张明华、唐明辉、李萌、王妙俊、李明铭、田宁、黄盼盼、张鹏、王前程、陈钦宇、杜秋实、葛瑞琪、张瑞松、潘睿哲、王润基、陈瑞杰、金瑞麟、陈如意、陆尚昊、周尚彦、陈善煌、叶盛丰、王诗语、余水平、周顺峰、潘舒婷、李珊珊、周爽、吴绍庆、叶盛丰、云涛、裴天、孙天宇、王涛、曾望鼎、赵万佳、刘文、梁文峰、高文俊、余文钦、张文涛、肖文龙、安伟、刘小东、王笑寒、陈小康、聂晓涛、程鑫、刘欣、谢欣、刘兴超、杨新宇、Xinyuan Li、苏学成、林旭恒、李晓青、金相悦、沈晓金、陈晓莎、孙晓文、Xiaoxiang Wang、宋新楠、周心怡、王宪祖、单新霞、李永康、王永强、魏永祥、张阳、徐艳红、李尧、赵耀、孙耀峰、王耀辉、余毅、张一超、石一凡、熊一良、何颖、朴一石、王艺松、谭逸轩、马一扬、刘一元、郭永强、欧源、王玉端、宫悦、邹宇恒、Yujia He、熊云帆、罗宇翔、游宇翔、刘宇轩、周宇阳、朱永祥、徐艳红、黄艳萍、李耀辉、郑毅、朱雨辰、马云贤、唐颖、查宇坤、闫雨婷、任泽中、任泽辉、沙章立、付哲、徐哲安、谢振达、张正言、郝哲文、马志成、严志刚、吴志宇、顾子辉、Zijia Zhu、刘子俊、李子林、谢紫薇、宋子阳、潘子正、黄震、徐志鹏、张仲宇、张震。2025. [DeepSeek-R1: 通过强化学习激励大语言模型的推理能力](#)。预印本, arXiv:2501.12948.

深度求索，刘爱新，冯蓓，薛冰，Bingxuan Wang，吴博超，陆成达，赵成刚，邓承启，张宸宇，阮冲，戴大麦，郭达雅，杨德建，陈德礼，季东杰，李尔航，林方韵，戴福聪，

Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, , 以及等人。2024年。[Deepseek-v3 技术报告](#)。预印本, arXiv:2412.19437。

Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Alan Huang, Songyang Zhang, Kai Chen, Zhixin Yin, Zongwen Shen, Jidong Ge, and Vincent Ng. 2024年。[LawBench: 评估大语言模型法律知识的基准](#)。收录于2024年自然语言处理经验方法会议论文集, 第7933–7962页, 美国佛罗里达州迈阿密。计算语言学协会。

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, 以及等人。2024年。[Llama 3 模型群](#)。预印本, arXiv:2407.21783。

Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiaoliu, Jun Zhao, and Kang Liu. 2024。[DA-code: 面向大语言模型的智能体数据科学代码生成基准](#)。载于2024年自然语言处理经验方法会议论文集, 第13487–13521页, 美国佛罗里达州迈阿密。计算语言学协会。

Shadi Iskander, Sofia Tolmach, Ori Shapira, Nachshon Cohen, and Zohar Karnin. 2024。[质量至关重要: 评估工具使用型大语言模型的合成数据](#)。载于2024年自然语言处理经验方法会议论文集, 第4958–4976页, 美国佛罗里达州迈阿密。计算语言学协会。

John Jaihar, Neehal Lingayat, Patel Sapan Vijaybhairav, Gautam Venkatesh, and K. P. Upla. 2020。[使用机器学习算法的智能家居自动化](#)。载于2020年新兴技术国际会议 (INCET), 第1–4页。

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023年。[Mistral 7b](#)。预印本, arXiv:2310.06825。

Chung-Wah (Chloe) Ki, Erin Cho 和 Jung-Eun Lee. 2020年。[智能个人助理 \(IPA\) 能成为你的朋友吗? IPA 与用户之间的准友谊发展机制](#)。计算机与人类行为, 111:106412。

Evan King, Haoxiang Yu, Sangsu Lee 和 Christine Julien. 2024年。[Sasha: 利用大语言模型在智能家居中进行创造性目标导向推理](#)。[ACM交互式移动可穿戴普适技术汇刊](#), 8(1)。

Sunjun Kweon, Yeonsu Kwon, Seonhee Cho, Yohan Jo, and Edward Choi. 2023年。[Open-WikiTable: 用于开放域问答及表格复杂推理的数据集](#)。载于 *Findings of the Association for Computational Linguistics: ACL2023*, 第8285–8297页, 加拿大多伦多。Association for Computational Linguistics。

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020年。[面向知识密集型NLP任务的检索增强生成](#)。载于《*神经信息处理系统进展*》, 第33卷, 第9459–9474页。Curran Associates公司。

Qiwei Li, Zuchao Li, Ping Wang, Haojun Ai, and Hai Zhao. 2024年。[面向文档语义实体识别的基于超图的理解](#)。载于《第62届计算语言学协会年会论文集 (第一卷: 长论文)》, 第2950–2960页, 泰国曼谷。计算语言学协会。

Zeming Liu, Haifeng Wang, Zheng-Yu Niu, Hua Wu, Wanxiang Che, and Ting Liu. 2020年。[迈向多类型对话上的会话式推荐](#)。载于《第58届计算语言学协会年会论文集》, 第1036–1049页, 线上。计算语言学协会。

Ewa Luger 和 Abigail Sellen. 2016年。“[就像有一个非常糟糕的私人助理](#)”: 用户对对话代理的期望与体验之间的鸿沟。载于2016年CHI人机交互系统会议论文集, CHI '16, 第5286–5297页, 美国纽约州纽约市。美国计算机协会。

OpenAI, :, Ahmed El-Kishky, Alexander Wei, Andre Saraiva, Borys Minaiev, Daniel Selsam, David Dohan, Francis Song, Hunter Lightman, Ignasi Clavera, Jakub Pachocki, Jerry Tworek, Lorenz Kuhn, Lukasz Kaiser, Mark Chen, Max Schwarzer, Mostafa Rohaninejad, Nat McAleese, o3贡献者, Oleg Mürk, Rhythm Garg, Rui Shu, Szymon Sidor, Vineet Kosaraju 和 Wenda Zhou. 2025年。[使用大型推理模型进行竞技编程](#)。预印本, arXiv:2502.06807。

Paul Jasmin Rani, Jason Bakthakumar, B. Praveen Kumar, U. Praveen Kumar, and Santhosh Kumar. 2017。[使用自然语言处理 \(NLP\) 和物联网 \(IoT\) 的语音控制家庭自动化系统](#)。收录于2017年第三届国际科学技术工程与管理会议 (ICONSTEM), 第368–373页。

Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Xueli Liu, and Gregory Dudek. 2024. [通过自主大语言模型智能体实现的](#)

[AIoT智能家居](#)。IEEE物联网期刊, 第1-1页。

史应天、刘晓艺、Chun Yu、杨天傲、高程、梁晨、史元春。2024a. [弥合智能家居中自然用户表达与复杂自动化编程之间的鸿沟](#)。预印本, arXiv:2408.12687。

石正良、张硕、孙伟伟、高深、任鹏杰、陈竹敏、任昭春。2024b. [多跳问答中检索增强生成的先生成后落地策略](#)。载于第62届计算语言学协会年会论文集（第一卷：长论文），第7339-7353页，泰国曼谷。计算语言学协会。

John Sweller. 2011. [第二章 - 认知负荷理论](#)。载于学习与动机心理学第55卷, 第37-76页。学术出版社。

Gemma团队、Morgane Riviere、Shreya Pathak、Pier Giuseppe Sessa、Cassidy Hardin、Surya Bhupathi- raju、Léonard Hussenot、Thomas Mesnard、Bobak Shahriari、Alexandre Ramé、Johan Ferret、Peter Liu、Pouya Tafti、Abe Friesen、Michelle Casbon、Sabela Ramos、Ravin Kumar、Charline Le Lan、Sammy Jerome、Anton Tsitsulin、Nino Vieillard、Pi-otr Stanczyk、Sertan Girgin 等人。2024年。Gemma 2: 在实用尺寸上改进开源语言模型。预印本, arXiv:2408.00118。

Qwen团队。2025. [Qwq-32b: 拥抱强化学习的力量](#)。

王恒毅、石海舟、谭世伟、秦伟一、王文渊、张屯玉、Akshay Nambi、Tanuja Ganu、王浩。2024a. [多模态大海捞针: 评测多模态大语言模型的长上下文能力](#)。预印本, arXiv:2406.11230。

王宏儒、王睿、薛博洋、夏鹤鸣、曹景涛、Zeming Liu、潘杰夫和Kam-Fai Wong。2024b. [AppBench: 针对复杂用户指令规划来自各类应用程序的多个应用程序接口](#)。收录于2024年自然语言处理经验方法会议论文集, 第15322-15336页, 美国佛罗里达州迈阿密。计算语言学协会。

Huiming Wang、Liyang Cheng、Wenxuan Zhang、De Wen Soh和Lidong Bing。2024c. [顺序无关的数据增强用于少样本命名实体识别](#)。收录于计算语言学协会第62届年会论文集（第一卷：长论文），第7792-7807页，泰国曼谷。计算语言学协会。

Ke Wang、Jiahui Zhu、Minjie Ren、Zeming Liu、Shiwei Li、Zongye Zhang、Chenkai Zhang、Xiaoyu Wu、Qiqi Zhan、Qingjie Liu、and Yunhong Wang. 2024年d. [A](#)

[关于大语言模型数据合成与增强的综述](#)。预印本, arXiv:2410.12896。

Zheng Wang、Shu Teo、Jieer Ouyang、Yongjun Xu和Wei Shi. 2024e. [M-RAG: 通过多分区检索增强生成强化大语言模型性能](#)。收录于计算语言学协会第62届年会论文集（第一卷：长论文），第1966-1978页，泰国曼谷。计算语言学协会。

An Yang、Baosong Yang、Beichen Zhang、Binyuan Hui、Bo Zheng、Bowen Yu、Chengyuan Li、Dayiheng Liu、Fei Huang、Haoran Wei、Huan Lin、Jian Yang、Jianhong Tu、Jianwei Zhang、Jianxin Yang、Jiaxi Yang、Jingren Zhou、Junyang Lin、Kai Dang、Keming Lu、Keqin Bao、Kexin Yang、Le Yu、Mei Li、Mingfeng Xue、Pei Zhang、Qin Zhu、Rui Men、Runji Lin、Tianhao Li、Tingyu Xia、Xingzhang Ren、Xuancheng Ren、Yang Fan、Yang Su、Yichang Zhang、Yu Wan、Yuqiong Liu、Zeyu Cui、Zhenru Zhang 和 Zihan Qiu. 2024年。Qwen2.5 技术报告。arXiv 预印本 arXiv:2412.15115。

Ziqi Yin、Mingxin Zhang 和 Daisuke Kawahara. 2024年。Harmony: 一个基于本地部署大语言模型的、用于响应式管理和动作优化的家庭智能体。预印本, arXiv:2410.14252。

Haoliang Yu、Jie Hua、and Christine Julien. 2021. [分析 IFTTT 配方以研究人类如何使用物联网 \(IoT\) 设备](#)。载于第19届ACM嵌入式网络传感器系统会议论文集, SenSys '21, 第537-541页。ACM。

Xin Zeng、Xiaoyu Wang、Tengxiang Zhang、Chunyu Shengdong Zhao、and Yiqiang Chen. 2024. [GestureGPT: 面向零样本自由形式手势理解的大语言模型智能体](#)。Proc.ACM Hum.-Comput.Interact., 8(ISS)。

Bowen Zhang、Kehua Chang、and Chunping Li. 2024a. [增强生成式语言模型中句子嵌入的简单技术](#)。载于先进智能计算技术与应用, 第52-64页, 新加坡。施普林格·自然新加坡。

Wenxiao Zhang、Xiangrui Kong、Thomas Brauni and Jin B. Hong. 2024b. [Safeembodai: 具身AI系统中移动机器人的安全性框架](#)。预印本, arXiv:2409.01630。

张欣荣、陈英发、胡圣鼎、徐子航、陈俊豪、Moo Hao、韩旭、Zhen Thai、王硕、刘知远、孙茂松。2024c. [∞Bench: 将长上下文评估扩展至10万词元之外](#)。载于《第62届计算语言学协会年会论文集（第一卷：长论文）》，第15262-15277页，泰国曼谷。计算语言学协会。

张卓、胡祥静、张景源、张雅婷、HuiWang、曲立真和徐增林。2023年。FEDLEGAL: 首个现实世界的联邦学习

法律自然语言处理基准。收录于第61届计算语言学协会年会论文集（第一卷：长论文），第3492–3507页，加拿大多伦多。计算语言学协会。

周晓玲、叶伟、王一东、蒋超亚、李正、谢睿和张世坤。2024年。通过隐式演示增强提升上下文学习能力。载于第62届计算语言学协会年会论文集（第一卷：长论文），第2810–2828页，泰国曼谷。计算语言学协会。

朱子豪、吴秉哲、张正友、韩磊、刘青山和吴宝元。2024年。Earbench: 评估基于基础模型的具身人工智能体任务规划的物理风险意识。预印本，arXiv:2408.04449。

A 附录



图6: HomeBench中的所有房间类型。

A.1 数据收集

表6展示了我们使用ChatGPT-3.5为用户指令生成的提示词。为了提高生成指令的质量和准确性，我们在提示词中加入了两个示例案例。这些示例为模型提供了参考，帮助其更好地理解任务要求，并生成更精确、更符合上下文的用户指令响应。

图6展示了我们设计的具有不同功能的多个房间。每个房间的功能-

能都经过精心规划，以确保其符合现实使用场景，涵盖广泛的智能家居应用。这种结构化的设计使模型能更有效地处理多样化的现实世界情境。

图7展示了一个虚拟房间构建示例，我们演示了如何根据预定义规则选择和配置智能家居设备，以确保一致性和逻辑兼容性。如代码所示，客厅设置中的某些设备存在互斥约束。例如，空调、加热器和风扇不能共存于同一空间，以防止功能冲突。同样，加湿器和除湿器也互斥，以维持适当的环境控制。对于其他设备类别，如照明、电视和音频系统，我们从预定义列表中随机选择设备，并决定是否将其添加到当前房间。如果房间已包含设备，新的选择可以作为扩展添加；如果房间为空，则可以初始化一组新设备，从而创建多样化的虚拟房间配置。这种构建逻辑不仅确保了设备的合理选择，也为进一步的智能家居场景模拟和优化奠定了基础。

表7列出了我们数据集中包含的所有设备类别，以及每个类别可能对应的操作方法。为了增强设备多样性并确保与各类智能家居品牌的多功能特性兼容，我们为每个设备分配了一套基本功能（例如，开关），并随机添加了额外功能。这种方法使设备能够动态适应不同情况，提供更灵活、更真实的用户体验。

图8展示了100个虚拟家居环境中设备分布与类型的统计概览。在12个功能各异的房间内，用户可与至少47个可操作设备进行交互，这些设备涵盖10个不同的设备类别。这种全面的设置确保了多样且实用的测试环境，有效模拟了现实世界智能家居场景。

A.1.1 指令质量

在选择指令生成方法时，我们进行了一项系统的对比研究。实验设计涵盖了两个关键维度。首先，我们比较了几种大语言模型的指令生成能力。

提示词

你是一个人人工智能，擅长将机器指令转换为由人类发出的命令。语气不必客气，直接给出指令，并使用口语化的表达格式。如果给出多条机器指令，你需要将它们转换成一个句子。如果指令包含“解释”字段，请注意生成幅度调整类型的人类指令。以下是一些示例：

机器指令：{ 'instruction' : ' set_intensity(30)' , ' device' : ' humidifier' , ' explain' : ' increase 2percent' , ' room' : ' guest_bedroom' }

人类指令：将客房中加湿器的强度提高2%。

机器指令：{ 'instruction' : ' set_temperature(26)' , ' device' : ' air_conditioner' , ' explain' : ' decrease1 percent' , ' room' : ' guest_bedroom' }

人类指令：将客房中的空调温度降低1度。

以下是您想要修改的机器指令：

表6：用于提示大语言模型生成用户指令的提示词。在该提示词中，我们提供了两个示例，以更清晰地说明不同的调整方法。第一个示例演示了如何为目标准确控制直接设置特定值。相比之下，第二个示例采用了相对调整方法，基于当前状态修改数值以实现动态调谐。

设备类型	方法
灯光设备	打开_；关闭_；设置_亮度；设置_颜色
空调设备	打开_；关闭_；设置_温度；设置_模式；设置_风扇_速度；设置_摆动
加热设备	打开_；关闭_；设置_温度；设置_模式；设置_风扇_速度
风扇设备	打开_；关闭_；设置_速度；设置_摆动
车库门设备	打开；关闭
百叶窗设备	打开；关闭
窗帘设备	打开；关闭；设置_开合度
空气净化器设备	打开_；关闭_；设置_模式；设置_风扇_风速
热水器设备	打开_；关闭_；设置_温度；设置_模式
媒体播放器设备	播放；暂停；停止；设置_音量；设置_歌曲；设置_艺术家；设置_风格
扫地机器人设备	开始,暂停;停止;充电;设置_模式;设置_清洁_区域
香薰设备	打开_;关闭_;设置_强度;设置_间隔
垃圾桶设备	pack
加湿器设备	打开_;关闭_;设置_强度;设置_模式
除湿器设备	打开_;关闭_;设置_强度;设置_模式

表7：HomeBench中所有设备类型及其方法。

```

1  class 视觉客厅:
2      def __初始化__(自身) -> None:
3          self.name = "客厅_"
4          self.设备 = []
5          self.不存在的设备_ = []
6          self.设备.添加(随机.选择(灯光设备列表)("开
            "))
7          如果随机.随机数() > 0.5:
8              self.devices.append(random.choice(
                空调设备列表)("开启"))
9              self.unexist_devices.append(random.choice(
                HeatingDeviceList)("开"))
10             self.unexist_devices.append(random.choice(
                FanDeviceList)("开"))
11         否则:
12             self.devices.append(random.choice(
                HeatingDeviceList)("开"))
13             self.devices.append(random.choice(FanDeviceList)("
                on"))
14             self.unexist_devices.append(random.choice(
                空调设备列表)("开启"))
15
16         random.choice([self.devices, self.unexist_设备]).
            append(random.choice(CurtainDeviceList)("on"))
17         random.choice([self.devices, self.unexist_设备]).
            append(random.choice(AirPurifiersDeviceList)("开启"))
18
19         如果随机.随机() > 0.5:
20             self.devices.append(random.choice(
                加湿器设备列表)("开启"))
21             self.unexist_devices.append(random.choice(
                除湿器设备列表)("开启"))
22         否则:
23             self.devices.append(random.choice(
                除湿器设备列表)("开启"))
24             self.unexist_devices.append(random.choice(
                加湿器设备列表)("开启"))
25
26         random.choice([self.devices, self.unexist_devices]).
            append(random.choice(AromatherapyDeviceList)("开启"))
27         random.choice([self.devices, self.unexist_devices]).
            append(random.choice(TrashDeviceList)("on"))
28         random.choice([self.devices, self.unexist_devices]).
            append(random.choice(MediaPlayerDeviceList)("play"
                ))
29         random.choice([self.devices, self.unexist_devices]).
            append(random.choice(PetFeederDeviceList)("on"))
30
31         self.random_initialize()
32         self.state = self.get_status()

```

图 7：智能家居虚拟房间配置：确保逻辑设备选择。

提示词

你是‘AI’，一个控制房屋内设备的有用人工智能助手。请按照指示完成以下任务，或仅根据所提供的信息回答以下问题。设备当前状态及其拥有的方法如下所示，请仅使用提供的方法。操作不存在的属性和设备时，输出`error\_input`。仅输出机器指令，并将其用`{}`括起来。

<家庭_状态>
 以下提供当前每个房间中所有设备的状态。
 家庭中每个房间所有设备的状态、每个设备的可调属性及其阈值。
 可调整属性：
（家庭设备状态，表9 展示了一个示例。）
 </home_状态>

<设备_方法>
 以下提供了控制当前房屋中每个设备的方法-
 hold:
(设备方法，表10 展示了一个示例。)
 </device_method>

以下是您需要回复的用户指令。
 <用户指令：>
 用户指令
 <机器指令：>

表8：用于提示大语言模型生成操作的提示词。蓝色是初始状态，橙色是设备方法

d.

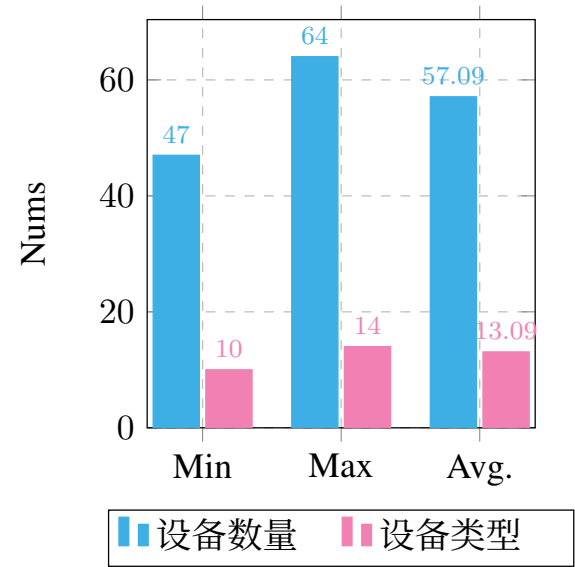


图8：100个家庭环境中设备数量和类型的统计。

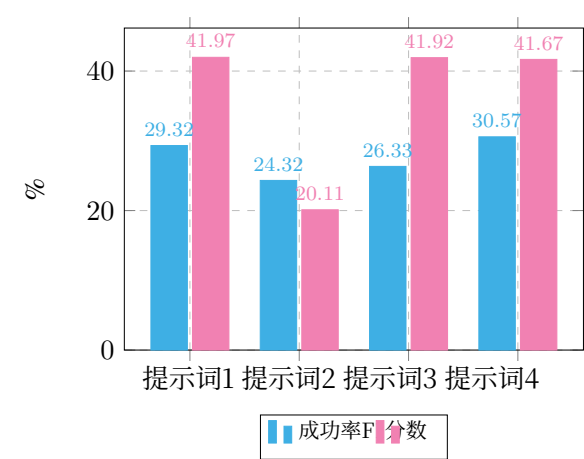


图9：Gemma2-9B-Instruct在不同提示词下在测试数据集上的性能。提示词1和提示词2是手动编写的。提示词3是通过使用GPT-4修改提示词1生成的。提示词4也是手动编写的，是我们最终使用的提示词。

示例	
主	-卧室:
-灯	-状态: 关闭
-亮度	-亮度: 35 (范围: 0 - 100)
-颜色	-颜色: [232, 50, 198]
-供暖	-状态: 关闭
-模式	-模式: 仅风扇 (选项['供暖' , '仅风扇_'])
-风扇_速度	-风扇_速度: 自动 (选项['自动' , '低' , '中' , '高'])
-fan	-状态: 开启
-摆动	-摆动: 自动 (选项['自动' , '向上' , '中间' , '向下'])

表9：家庭设备状态示例。

示例	
主_卧室.灯.打开_();	
主_卧室.灯.关闭_();	
主_卧室.灯.设置_亮度(亮度:整数);	
主_卧室.灯.设置_颜色(颜色:类型.元组[整数, 整数, 整数]);	
主_卧室.供暖.打开_();	
主_卧室.供暖.关闭_();	
主_卧室.供暖.设置_模式(模式:字符串);	
主_卧室.供暖.设置_风扇_速度(风扇_速度:字符串);	
主_卧室.风扇.打开_();	
主_卧室.风扇.关闭_();	
主_卧室.风扇.设置_摆动(摆动:str);	

表10：家居设备方法示例。

日期类型	用户指令	输出
VS	将阳台上的灯亮度设置为50。	阳台.灯.set_亮度(50)
IS	暂停客厅里的媒体播放器。	error_input
VM	将客房的空调温度降低到20度, 将门厅的灯亮度设置为50, 并将书房的除湿器强度提高32%。	guest_卧室.air_conditioner.set_温度(20), foyer.light.set_brightness(50), study_房间.除湿器.set_强度(70)
IM	将书房的加湿器强度设置为60, 将阳台上的媒体播放器音量调整至60, 并将阳台上的窗帘开合度设置为20。	错误_输入, 错误_输入, 错误_输入
MM	将客房的空调温度设置为16度, 关闭书房里的灯, 将阳台上的媒体播放器音量降低30%, 将主卧室的除湿器强度设为60, 并将浴室的供暖温度调整到27度。	guest_卧室.空气_调节器.设置_温度(16), study_房间.灯.turn_off(), 错误_输入, 错误_输入, 浴室.供暖.设置_温度(27)

表11：HomeBench中不同数据类型的示例。

(包括ChatGPT-3.5、GPT-4和Qwen-72B)。其次，我们建立了一个由人工编写的指令组成的基准对照组。对于每种方法，我们生成了200个指令样本，并基于三个关键指标进行了全面评估：生成质量、生成效率和经济成本。

实验结果表明，在生成质量方面，ChatGPT-3.5的表现与人工编写的指令相当。然而，它在效率和成本效益方面都展现出显著优势。具体而言，ChatGPT-3.5平均仅需2分钟即可生成指令，成本低于0.5美元，而人工编写指令则耗时近4小时，成本约为25美元。

基于这些发现——质量相当，但效率和成本显著优越——我们最终选择ChatGPT-3.5作为我们的指令生成工具。

A.2 实验

实现细节。我们的实验结合使用了开源模型、闭源模型和微调模型，并根据需要利用不同的硬件和应用程序接口。对于开源模型，所有实验均在NVIDIA RTX 3090 GPU上进行，以确保高效的执行和评估。对于闭源模型和Qwen2.5-72B-Instruct，我们使用了OpenAI提供的应用程序接口，使我们能够在相同的实验条件下对它们的性能进行基准测试。微调实验在NVIDIA A800 GPU上进行，利用其高内存容量和计算能力来优化模型性能。我们采用Hugging Face的PEFT（参数高效微调）框架来高效地微调模型，同时最大限度地降低计算成本。如图所示10，我们使用LoRA来微调Qwen模型。

表8展示了所有模型测试中使用的提示词设计。每个提示词都包含房间设备的状态信息，以确保模型准确理解当前环境并生成适当的响应（见表9）。此外，我们列出了所有设备的调用方法，以便模型能正确执行相关操作，详见表10。

为了确定最合适的提示词，我们使用四种不同的提示词进行了实验。

```
1  lora_config = L 或 C 配置
2      r=16,
3      lora_alpha=32,
4      目标_模块=[
5          "q_proj", "k_proj",
6          "v_proj", "_项目",
7          "gate_proj", "
8          up_proj", "
          下载_项目"],
          lora_dropout=0.1,
          偏见="无",
          任务_类型="因果_语言模型"
8  )
```

图10：用于微调 Qwen2.5-7B-Instruct 模型的 LoRA 配置。

提示词1和提示词2是人工编写的，旨在探索人工设计提示词的有效性。提示词3源自提示词1，但经过GPT-4精炼，以评估大语言模型在提示词优化方面的能力。提示词4同样是人工编写的，并最终被选为最终的提示词版本。图9展示了不同提示词在测试数据集上的性能，比较了它们的有效性，并为选择最优提示词提供了见解。

A.2.1 少样本学习

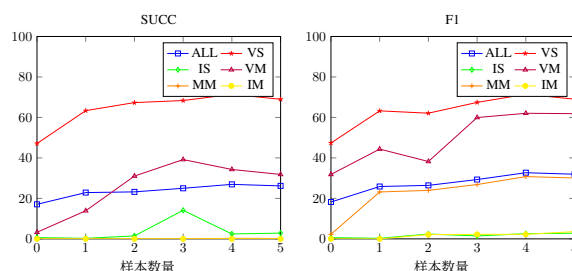


图11：Llama3-8B-Instruct模型在添加不同类型数据样本时的性能（添加顺序为：VS、IS、VM、MM、IM）。

表12展示了用于评估各种少样本学习方法的不同类别样本。这些样本涵盖了一系列设备功能类别，从上到下依次为：VS、IS、VM、MM和ME。通过纳入这些多样化的类别，我们能更有效地评估模型在不同现实世界智能家居场景中的表现。

少样本学习

以下是几个示例，你的输出格式应与示例中的结果保持一致：
示例中提供的：

<示例1>

<用户指令：> 将客房的空调温度调低
1度。

<机器指令：> {guest_卧室.空调.设置_温度(26)}

</示例1>

<示例2>

<用户指令：> 打开客厅的灯。

<机器指令：> {error_输入}（如果不存在设备，或者设备上没有可操作的
属性或方法，请输出为“error_输入”。）

</示例2>

<示例3>

<用户指令：> 将客房加湿器的强度设置为20%，
将书房除湿器的强度增加30%，
并将书房除湿器的强度设置为40%。

<机器指令：> {guest_bedroom.humidifier.set_intensity(20),study_room.de
加湿器.设置_强度(10),书房_房间.除湿器.设置_强度(40),}

</示例3>

<示例4>

<用户指令：> 将走廊的芳香疗法强度提高
4%，将车库的灯亮度设置为40，将客厅的
空调风扇速度调至低档，并将媒体播放器的音量设置为
客厅调至0。

<机器指令：> {corridor.aromatherapy.set_强度(30),错误_输入,错误_
输入,客厅_房间.媒体_播放器.设置_音量(0),}

</示例4>

<示例5>

<用户指令：> 打开车库里的空气净化器并提高音量
将客厅媒体播放器的音量降低50%。

<机器指令：> {error_输入,error_输入}

</示例5>

表12：不同类型的少样本示例，从上到下的顺序依次是视觉选择任务、是、虚拟机、多模态任务、即时消息。

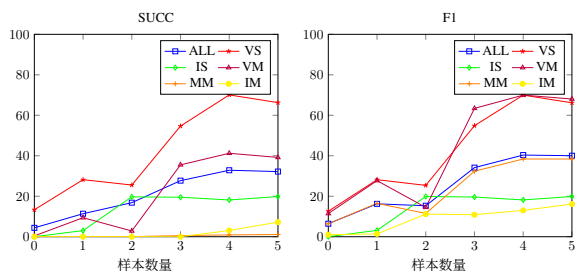


图12: Gemma2-9B-Instruct模型在添加不同类型数据样本时的性能表现（添加顺序为：VS、IS、VM、MM、IM）。

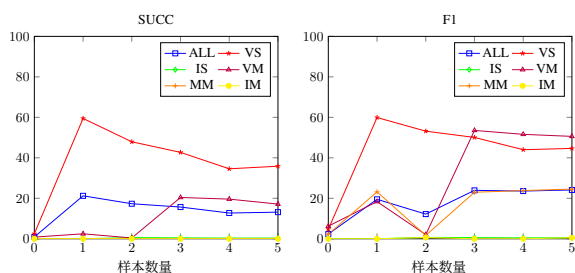


图13: Mistral-7B-v0.3模型在添加不同类型数据样本时的性能表现（添加顺序为：VS、IS、VM、MM、IM）。

图 11、12和13 分别展示了 Llama3-8B-Instruct、Gemma2-9B-Instruct 和 Mistral-7B-v0.3 在不同少样本设置下的性能表现。Llama3-8B-Instruct 和 Gemma2-9B-Instruct 的结果与我们在第 5.1 节中的发现一致：随着少样本示例数量的增加，模型性能通常有所提升，尽管提升程度因数据类型而异。然而，我们也观察到在某些任务中，特别是 IS 和 VS，随着样本数量的增加，性能反而下降。这种下降很可能是由于额外样本中缺乏家庭设备状态信息所致。由于新增的示例仅包含用户指令和设备操作，模型难以准确判断设备状态，从而导致混淆和性能下降。

有趣的是，Mistral-7B-v0.3 表现出一种独特的行为：它在仅使用 2-shot 时就达到了性能上限，超过此点后没有显著提升。这可能归因于我们未使用 Mistral 的指令调优版本，导致其较难完全理解我们任务的具体要求。这也解释了 Mistral 在主实验中相对较差的表现，进一步强化了我们的假设。

A.2.2 RAG设置与案例研究

我们根据房间类别对设备状态和方法进行了分段，并将这些分段输入到大语言模型中，以从最终层提取隐藏状态。这种方法使我们能够捕捉到设备功能和上下文依赖性的有意义的表征。

遵循 (Zhang等人, 2024a) 中概述的方法，我们在提示词中加入了“经过逐步思考后”这一短语。这一添加有助于模型更好地处理顺序推理并有效压缩语义信息，从而提升其对设备交互的理解。在获得分段文本块和用户指令的嵌入向量后，我们计算余弦相似度以促进检索。检索过程遵循以下条件：

- 如果相似度分数超过0.5且检索到的文本块数量大于3，则直接返回结果。
- 如果检索到的文本块少于3个，则返回相似度分数最高的前三个文本块，以确保模型拥有足够的上下文信息来准确执行。

此策略提升了检索效率，并确保模型拥有可靠且相关的设备上下文来进行决策。

在表13中，我们提供了不同数据类别下上下文检索错误的案例示例。这种上下文不匹配导致了一个反直觉的现象——在简单有效的任务上性能下降，但在更具挑战性的任务上却出现意外提升。经过进一步分析，我们发现根本原因在于设备状态/方法与用户指令之间的嵌入向量相似度不足，这阻碍了基于相似度的检索机制有效区分不同上下文片段的能力。为解决此问题，我们提出训练一个专门的检索优化模型，以提升上下文对齐准确率。

A.2.3 错误分析

表14详细列出了不同数据类型下各类错误类型的比例和数量。它展示了每种数据类型中不忠实性、上下文注意力错误和关键错误的分布情况，包括各自的百分比和绝对数量。这有助于更直观地分析错误模式。

数据类型	输入	黄金标准	生成
VS	将阳台上的灯亮度设置为50。阳台.灯.设置_亮度(50)	阳台.灯.设置_亮度(50)	error_输入
IS	暂停客厅的媒体播放器。	error_输入	error_输入
VM	将客房的空调温度调低至20度, 将门厅的灯亮度设置为50, 并将书房除湿器的强度提高32%。	guest_卧室.air-conditioner.set_温度(20), 门厅.灯.设置_亮度(50), 研究_房间.除湿器.设置_强度(70)	错误_输入, 错误_输入, error_输入,
IM	将书房加湿器的强度设为60, 将阳台媒体播放器的音量调至60, 并将阳台窗帘的开合度设为20。	error_输入, error_输入, 错误_输入,	错误_输入, 错误_输入, 错误_输入,
MM	将客房的空调温度设置为16度, 关掉书房里的灯, 将阳台媒体播放器音量降低30%, 将主卧室除湿器强度设为60, 并将浴室供暖温度调整至27度。	guest_卧室.air-conditioner.set_温度(16), study_房间.灯.turn_off(), error_input, 错误_输入, 浴室.供暖.设置_温度(27)	错误_输入, 错误_输入, 错误_输入, 错误_输入, 错误_输入,

表13： 当上下文中没有有用信息时的模型输出。红色表示错误输出，而绿色代表正确输出。le

Type	不忠实性	上下文注意力错误	关键错误
VS	50.53/805	7.53/120	41.93/668
IS	-	100/873	-
VM	53.83/71	60.90/81	44.36/59
IM	-	100/37	-
MM	23.45/732	44.44/1387	41.15/1253

未来工作的一个关键重点。

表14： 不同数据类型下各类错误类型的比例。前者为比率，后者为数量。

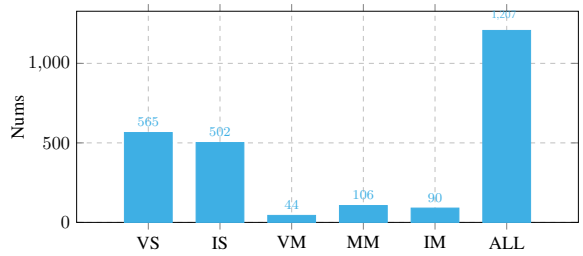


图14： 分布外数据分布。

可能影响模型在不同数据类型上性能的潜在问题。

A.2.4 分布外

我们创建了两个新的智能设备：床和宠物喂食器来构建分布外数据。其数据分布如图所示14。

A.2.5 推理延迟

在表15中，我们在RTX 3090平台上进行的测试表明，本地部署模型的推理延迟可以控制在数秒内，这通常能满足实际需求。然而，为了进一步提升用户体验——理想情况下将延迟降至一秒以下——提高推理速度仍是我们

模型	LLaMa3-8b	Qwen2.5-7B	Mistral-7B	Gemma2-9B	Deepseek-V3	GPT-4o
延迟	2.49秒	1.99秒	3.05秒	6.92秒	6.79秒	1.16秒

表15： 接口延迟